Lecture 2

The Unix Command Line

BMMB 852: Applied Bioinformatics

Why would one need Unix at all?

Unix is a bit like magic

Wingardium Leviosa - the levitation charm

- It does **wondrous** things, makes things fly 🐦‍🔥

Or

- Perhaps it **fizzles** out 🎇
- Perhaps it **blows** into your face 🌋
- Perhaps it turns **YOU** into a frog 🐸

Our goal is to have it do the first thing and not the others ...

What does UNIX bioinformatics look like?

Bacterial variant calling:

```
minimap2 -a -x sr -t $CPUS $REF $R1 $R2 \
 | samtools sort -l 0 --threads $CPUS \
 | bcftools mpileup -Ou -B --min-MQ 60 -f $REF - \
 | bcftools call -Ou -v -m - \
 | bcftools norm -Ou -f $REF -d all - \
 | bcftools filter -Ov -e 'QUAL<40 || DP<10 || GT!="1/1"'
 > variants.vcf
```

That's it.

Now "all" we need to understand what each command does ... 😃

What would you prefer?

```
minimap2 -a -x sr -t $CPUS $REF $R1 $R2 \
  | samtools sort -l 0 --threads $CPUS \
  | bcftools mpileup -Ou -B --min-MQ 60 -f $REF - \
  | bcftools call -Ou -v -m - \
  | bcftools norm -Ou -f $REF -d all - \
  | bcftools filter -Ov -e 'QUAL<40 || DP<10 || GT!="1/1"'
  > variants.vcf
```

or:



What would you prefer?

```
minimap2 -a -x sr -t $CPUS $REF $R1 $R2 \
  | samtools sort -l 0 --threads $CPUS \
  | bcftools mpileup -Ou -B --min-MQ 60 -f $REF - \
  | bcftools call -Ou -v -m - \
  | bcftools norm -Ou -f $REF -d all - \
  | bcftools filter -Ov -e 'QUAL<40 || DP<10 || GT!="1/1"'
  > variants.vcf
```

or



Well ... who wouldn't want the button? Gimme the button!

Except .... it is never about the commands! It is about the parameters.

```
minimap2 -a -x sr -t $CPUS $REF $R1 $R2 \
  | samtools sort -l 0 --threads $CPUS \
  | bcftools mpileup -Ou -B --min-MQ 60 -f $REF - \
  | bcftools call -Ou -v -m - \
  | bcftools norm -Ou -f $REF -d all - \
  | bcftools filter -Ov -e 'QUAL<40 || DP<10 || GT!="1/1"'
  > variants.vcf
```

The important things are `min-MQ 60`, `QUAL<40`, `DP<10` the effect of each may extremely impactful.

The commands are "well defined". Understanding the effects of each parameter on the other hand is far more complicated.

Biologists love the button



- The allure and promise of *easy to use*
- The illusion of not *needing to understand*.

Clarification/Disclaimer



It is not the button that is evil. It is fine to use if you understand what takes place at every step.

Catch 22: you cannot understand each step, unless you run them at command line.

Much damage is done to science by people pushing buttons, then publishing the results produce with buttons they don't understand.

Words (statements) are the most expressive way to describe complex events

Short statements are unambiguous, sharable, reusable

How does the command line work?

1. You type in "action" words that the computer executes.
2. You may combine these words to form longer commands.
3. You may place the results of these longer commands into files

---

These are universally useful skills - applicable to any domain of science.

Your "terminal" or "shell"

A little magical window into the "guts" of your computer

What is the "terminal"

Technically speaking there are two programs.

The terminal visualizes the command (colors and sizes of the fonts)

Inside the terminal we run the "shell", the command interface to the comptuer.

I will show commands typed into or printed by the terminal: `like this`

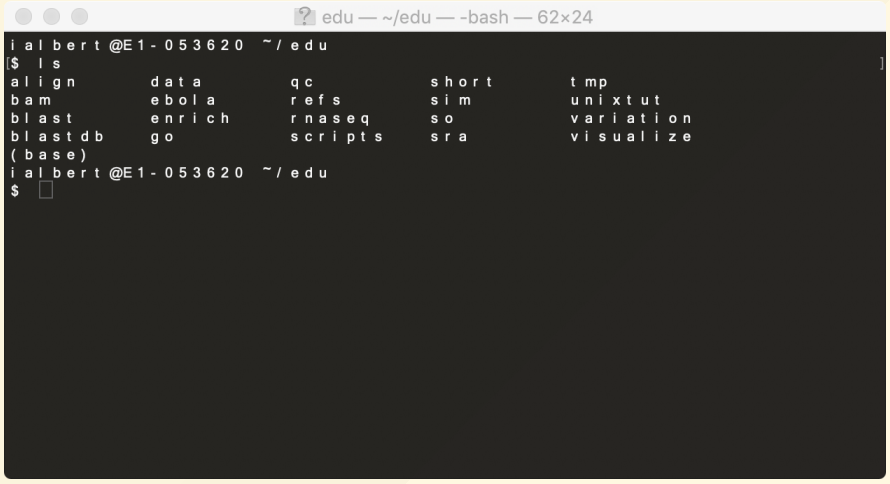
It is very important to become "comfortable" with your terminal.

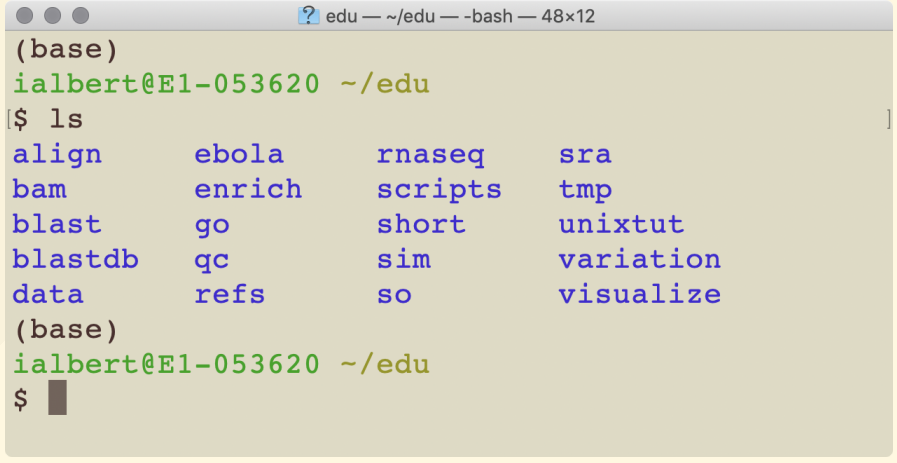
There is a "physical" aspect to it.

Make the fonts big.

Select a pleasing colors for fonts and backgrounds.

Compare the two





Customize your terminal

It makes a huge difference

in the speed you learn things

What is a filesystem?

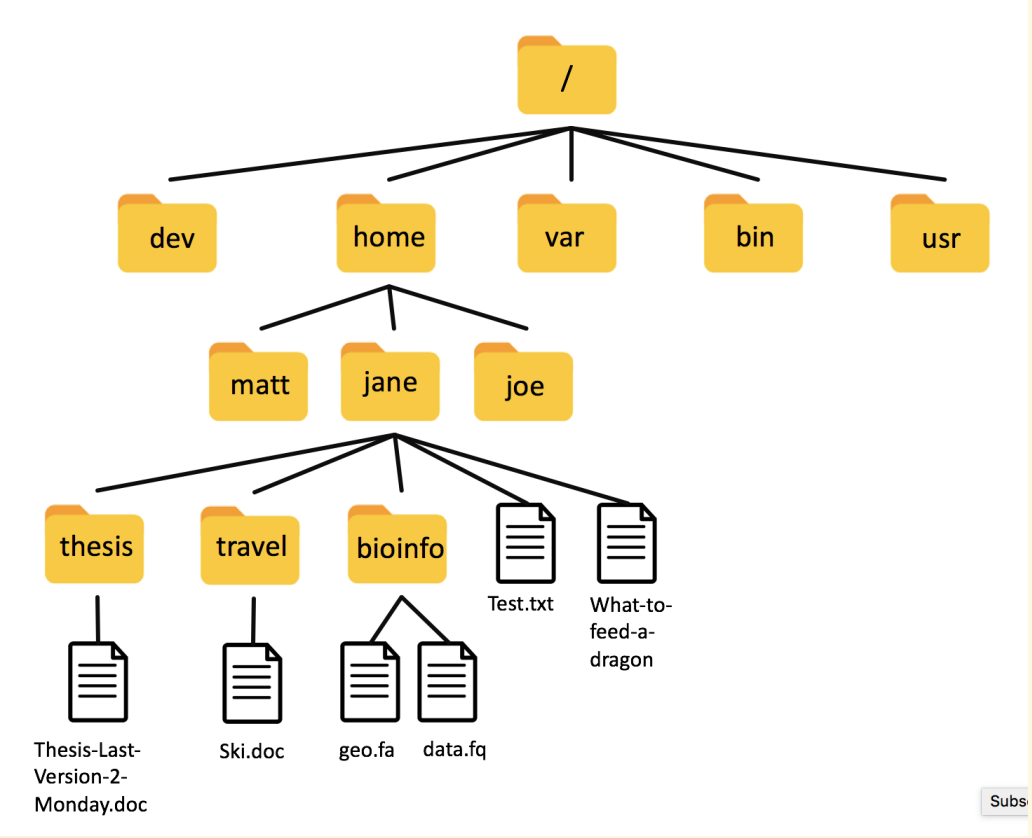
The "filesystem" is the term that describes how data is arranged on a computer.

The filesystem is made out of *directories* and *files*.

Windows started calling directories as *folders* - it is a synonim for *directory*.

A directory may contain files and other directories.

A filesystem example



Your terminal will start in **your home** directory.
Above it may be the `jane`, `matt` or `joe` based on login.

How to tell where you are

Print your current directory

```
pwd
```

for me it prints:

```
/Users/ialbert
```

What does it print for you?

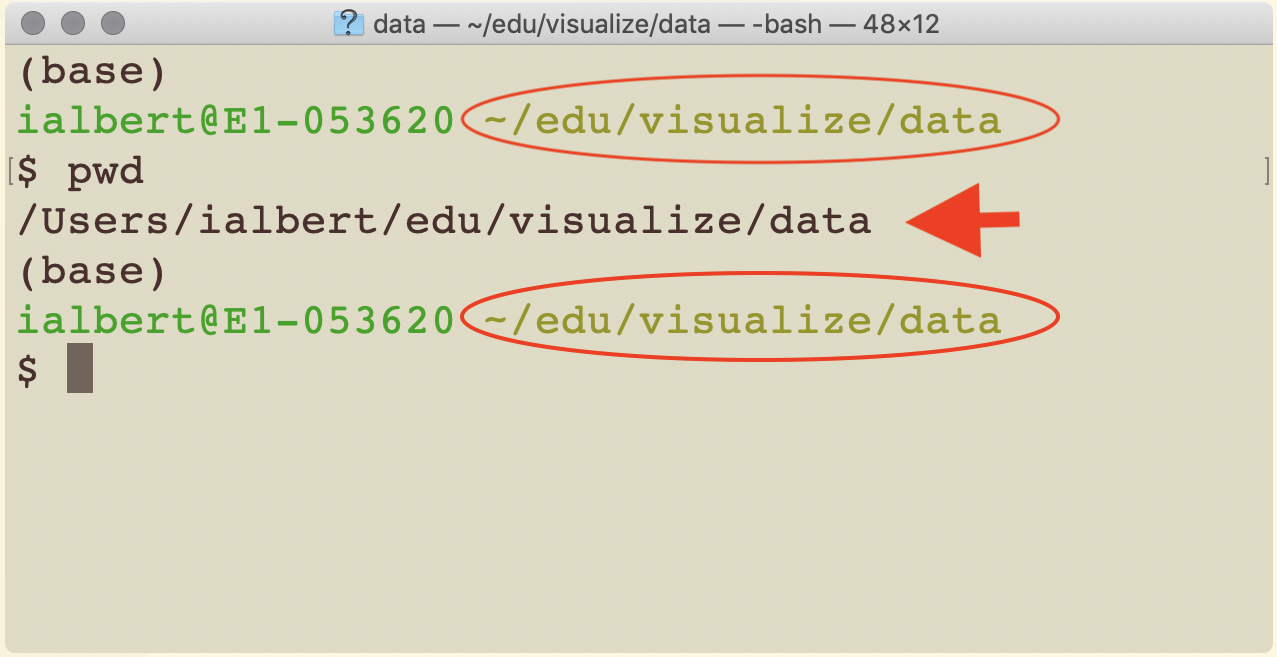
Always know where you are!

`pwd` is so important that the bash settings recommended by the book always show you the output of `pwd`



Get in the habit of using pwd to get used to the concept

every time you are in a directory type `pwd`



How do I get help on a command?

Type:

```
man
```

Output:

```
What manual page do you want?
```

Now try:

```
man ls
```

Press `q` to exit.

How do I know what commands I could be using?

Learn them from examples. Some are "logical" shorthand notations, others not so much.

- `ls` - list files
- `mv` - move (rename) files
- `rm` - remove files
- `cp` - copy files

A few more essential commands

- `cd` - change directory
- `pwd` - print working directory
- `mkdir` - make directory
- `rmdir` - remove directory

```
mkdir foo
cd foo
pwd
mkdir bar
cd
pwd
```

Where are you now? Why? How do you get to `bar`?

What kind of commands are there?

These usually come with your computer:

1. Shell commands: `cd`, `pwd`, ...
2. Operating system commands: `cp`, `mv` ...

These you will have to install yourself:

3. Domain specific specialized programs: `bwa`, `edirect`

What are command "flags"?

A flag is a small *decoration* used to customize the output of the tool. Compare the output of:

```
ls
```

to:

```
ls -l -h
```

The "flags" are `-l` and `-h`.

Flags usually may be combined into one as `-lh`

Everyone makes errors all the time

Using the command line can be error prone - a simple typo makes things crash - sometimes spectacularly but mostly pitifully.

We all make errors all the time - that is normal.

The difference between a novice and someone skilled is in the efficiency of their error recovery.

You'll get used to immediately recognizing and recovering from 95% of the errors.

What causes this fizzle 🎇?

Suppose you type

```
bwrapper
```

and you get the output 🎇:

```
-bash: bwrapper: command not found
```

Let's call this "Stupefy 1" - the stunning spell.

What caused this error? How do you solve it?

What causes this other fizzle 🎇?

Suppose you type

```
cat work/ranktest.txt
```

and you get the output:

```
cat: work/ranktest.txt: No such file or directory
```

Let's call this "Stupefy 2" - another stunning spell.

What caused this error? How do you solve it?

Stupefy 1 and Stupefy 2 are spells you will keep casting on yourself.

- `-bash: bwrapper: command not found`
- `work/ranktest.txt: No such file or directory`

Initially most of your errors will be these two errors.

You'll be "stupefyed" OMG it does not work. I don't understand why it does not work. 😩

Then later you make these errors often, you'll just recover from them much instantly, ah yes *I have just stupefyed myself*.

Do you know where you are? Do you know what is there?

Any time you run you get stuck make sure you know the answer the two questions in the title. You can find out those with

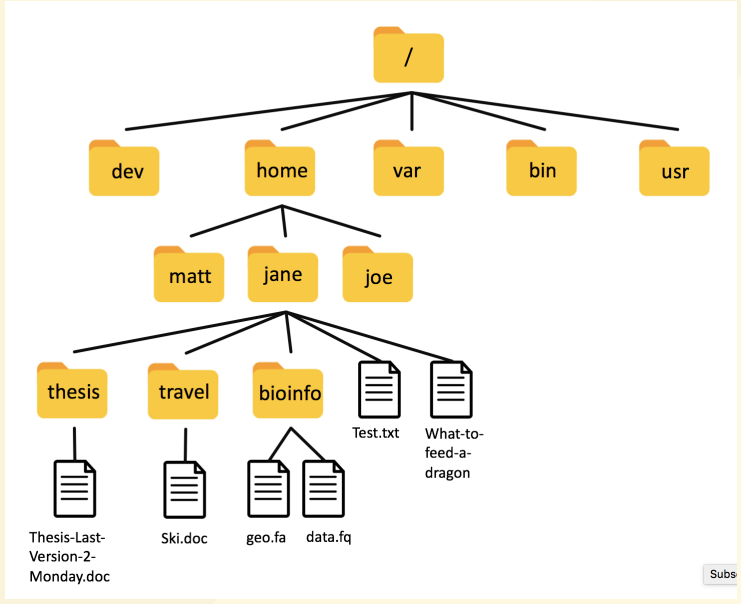
```
pwd
```

then

```
ls
```

You'll be suprised how often that helps you diagnose a problem.

Absolute paths

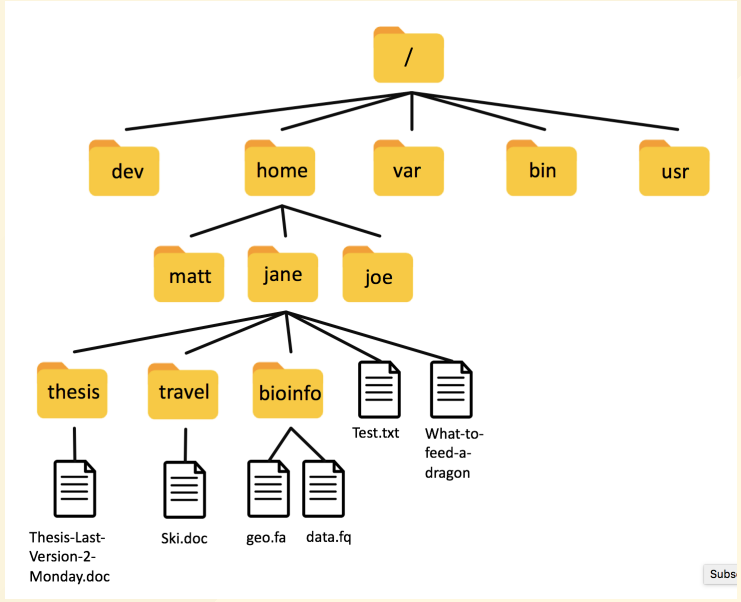


The location can be specified via a so called "path". Nodes of the tree are separated via the `/` character.

```
/home/jane/travel/Ski.doc
```

This is an *absolute path* to the file.

Relative paths



If you are in the `home/jane` directory then you could access the file with

```
travel/Ski.doc
```

There is no `/` at the start. This is a *relative* path.

Do you need more practice?

This lecture is not a full introduction to the command line.

Use online command line tutorials to further develop these skills.

Check the links for this lecture for further details.